10. Sets

Created by Li Huanyu, last modified by suncaper on Mar 09, 2026

Introduction

In the previous lesson, we learned about Dictionaries, which allow us to map Key-Value pairs to model complex data like JSON. But what if you don't need Key-Value pairs? What if you just have a massive list of data, and your primary concern is **removing duplicates** or **comparing** two different groups of data?

A **Set** is an unordered collection of **unique** elements. Think of it like a mathematical set. If you try to add an item to a set that is already there, Python simply ignores it.

In Data Engineering, Sets are your best friend for data cleaning and comparison. Whether you are finding the overlap between two database tables or removing duplicate log entries from a messy CSV, Sets handle these operations with incredible speed.

In this lesson, we will cover:

- Creating Sets and their core superpower: Deduplication.
- Mathematical Set Operations: Union, Intersection, and Difference.
- Why Sets are critical for ETL performance and fast lookups.

Part 1: The Core Feature (Deduplication)

Sets are created using curly braces `{}`, similar to dictionaries, but they do not use Key-Value pairs (no colons). Alternatively, you can convert any List into a Set using the `set()` function.

1. Creating and Deduplicating

The most common use case for a set in an ETL pipeline is converting a messy list containing duplicates into a clean set.

```
# A list of user IDs scraped from web logs (contains duplicates)
raw_log_ids = ["user_1", "user_2", "user_1", "user_3", "user_2"]

# Convert the list to a set to instantly remove duplicates
unique_ids = set(raw_log_ids)

print(unique_ids)
# Output: {'user_1', 'user_2', 'user_3'} (Note: order is not guaranteed!)

# Convert back to a list if you need to access by index later
clean_list = list(unique_ids)
```

2. Adding and Removing Sets are mutable, meaning you can add or remove items, but you cannot access them via an index number (e.g., `unique_ids[0]` will throw an error) because sets have no fixed order.

```
active_users = {"Alice", "Bob"}

active_users.add("Charlie")
active_users.add("Alice") # Ignored! Alice is already in the set.
```

```
print(active_users) # Output: {'Alice', 'Bob', 'Charlie'}
```

Part 2: Set Operations (Data Comparison)

Because Sets are based on mathematics, Python provides built-in operators to compare two sets. If you know SQL, these concepts will feel very familiar.

1. Intersection `&` (The "INNER JOIN" Concept)

Finds elements that exist in **both** sets.

```
table_a_keys = {"id_1", "id_2", "id_3"}
table_b_keys = {"id_2", "id_3", "id_4"}

# Find keys present in both tables
matching_keys = table_a_keys & table_b_keys
print(matching_keys)  # Output: {'id_2', 'id_3'}
```

2. Difference `-` (The "LEFT ANTI JOIN" Concept)

Finds elements that exist in the first set, but **not** the second. Highly useful for finding new records.

```
yesterday_users = {"Alice", "Bob", "Charlie"}
today_users = {"Alice", "Bob", "Charlie", "David"}

# Who is new today? (In today_users, but NOT in yesterday_users)
new_users = today_users - yesterday_users
print(new_users)  # Output: {'David'}
```

3. Union `|` (The "FULL OUTER JOIN" Concept) Combines all elements from both sets, ensuring the final result has no duplicates.

```
q1_customers = {"Alice", "Bob"}
q2_customers = {"Charlie", "Bob"}

# All unique customers for the first half of the year
all_customers = q1_customers | q2_customers
print(all_customers) # Output: {'Alice', 'Bob', 'Charlie'}
```

Part 3: The "Big Data" Context (Why do we care?)

Why not just use a `for` loop to check for duplicates in a List? **Speed.**

1. **$O(1)$ Time Complexity for Lookups:** In Big Data, checking if an item exists in a List containing 10 million rows requires Python to scan the list one by one (slow). Checking if an item exists in a Set takes virtually zero time, regardless of how large the Set gets. If you are comparing massive datasets, using a Set is mandatory to prevent your pipeline from taking hours to run.
2. **Spark `distinct()`:** When you write `df.distinct()` in PySpark to remove duplicate rows, the distributed engine is conceptually utilizing set logic across the cluster to filter out repeated data.

3. **Data Quality Checks:** Before joining two massive tables, Data Engineers often extract the primary keys into two sets and use the difference operator (`-`) to instantly verify if there are orphaned records that will fail the join.

Part 4: Hands-on Exercise (ETL Delta Processing)

Let's simulate a "Delta" (change) processing step. We have a master database of existing products and a new daily CSV file of products. We need to figure out which products to `INSERT` (new) and which to `UPDATE` (existing).

1. Open PyCharm.
2. Create a new file named `set_practice.py`.
3. Type the following code:

```
# ---------------------------------------------------------
# Big Data Simulation: Delta Load Processing
# ---------------------------------------------------------

"""
Scenario: We are comparing yesterday's master product list against
today's incoming data feed to route records to the correct database table.
"""

# 1. EXTRACT: Simulated data feeds
# Existing database records
master_db_ids = {"prod_100", "prod_101", "prod_102"}

# Incoming daily batch (Notice "prod_101" is duplicated in the raw feed!)
daily_batch_raw = ["prod_101", "prod_103", "prod_104", "prod_101"]

# 2. TRANSFORM: Clean the daily batch
daily_batch_clean = set(daily_batch_raw)

# 3. APPLY LOGIC: Set Operations
# Which products are completely new? (In daily batch, but not in master)
records_to_insert = daily_batch_clean - master_db_ids

# Which products already exist and need an update? (In both)
records_to_update = daily_batch_clean & master_db_ids

# 4. LOAD / OUTPUT: Log the routing decisions
print("--- ETL Routing Report ---")
print("New Records to INSERT: " + str(records_to_insert))
print("Existing Records to UPDATE: " + str(records_to_update))



Run the code. Output:

--- ETL Routing Report ---
New Records to INSERT: {'prod_103', 'prod_104'}
```

```
Existing Records to UPDATE: {'prod_101'}
```

What happens?

- We instantly deduplicate the incoming daily batch by wrapping the list in `set()`.
- We use the Difference operator (`-`) to isolate completely new product IDs that require `INSERT` commands.
- We use the Intersection operator (`&`) to isolate overlapping IDs that require `UPDATE` commands.

Summary

- **Sets use `{}`** but only contain single values, not Key-Value pairs.
- **Sets guarantee Uniqueness:** They automatically discard duplicate values.
- **Mathematical Operations:** Use `&` (Intersection), `-` (Difference), and `|` (Union) to compare datasets instantly.
- **Data Pipelines depend on Sets:** Because looking up values in a Set is exponentially faster than a List, Sets are the go-to tool for deduplication, delta processing, and data validation in ETL scripts.

No labels